

**Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Московский государственный университет
имени М.В.Ломоносова»**

**Казахстанский филиал
Направление 01.04.01 «Прикладная математика и
информатика»
Магистерская программа «Вычислительные технологии и
методы искусственного интеллекта»**

Токанов Расул Болатулы

**Интеграция онтологий в архитектуру генерации с
дополненной выборкой
для повышения точности семантического поиска и генерации
ответов**

Выпускная квалификационная работа

Научный руководитель:

Доцент кафедры, канд. физ.-мат. наук

Илья Николаевич Смирнов

Допустить к защите:

заведующий кафедрой

Л.В.Крицков

(подпись зав кафедрой)

«_____» _____ 2026 г.

Астана, 2026

Введение

Большие языковые модели на основе архитектуры Transformer продемонстрировали впечатляющие результаты в задачах обработки естественного языка. Однако практическое применение выявило фундаментальные ограничения: галлюцинации, невозможность обновления знаний без повторного обучения и отсутствие механизмов верификации ответов [1, 15].

Для преодоления этих ограничений предложена архитектура генерации с дополненной выборкой (RAG), в которой генеративная модель получает релевантные фрагменты из внешней базы знаний [2]. Несмотря на существенное улучшение качества, стандартный RAG обладает собственными ограничениями: потеря семантической связности при сегментации [5], отсутствие структурного знания о связях между понятиями [4, 6] и снижение релевантности при увеличении контекста [1].

Перспективным направлением является интеграция формальных онтологий в конвейер RAG. Работы OG-RAG [4] и OWL-RAG [7] продемонстрировали значительный потенциал: повышение полноты извлечения до 55 % и сокращение контекста на 27 %. Однако открытым остаётся вопрос об условиях эффективности онтологического расширения: не все запросы одинаково выигрывают от графового обогащения.

Целью работы является исследование возможностей улучшения архитектуры RAG путём интеграции онтологического слоя и адаптивной маршрутизации запросов, а также экспериментальное определение условий эффективности онтологического расширения.

Для достижения поставленной цели необходимо решить следующие задачи. Во-первых, провести анализ существующих подходов к интеграции онтологий в архитектуру RAG. Во-вторых, спроектировать модульную архитектуру OntoRAG с уровнями извлечения, онтологического расширения, ре-ранжирования и маршрутизации. В-третьих, разработать эвристический классификатор запросов для определения целесообразности использования онтологии. В-четвёртых, реализовать программный прототип с поддержкой шести конфигураций для абляционного исследования. В-пятых, провести экспериментальное исследование с использованием пяти метрик и анализа чувствительности. В-шестых, выполнить анализ результатов, выявить причины деградации и сформулировать рекомендации.

Научная новизна работы состоит в том, что предложена модульная архитектура OntoRAG v3.1 с изолированными компонентами, допускающими независимое абляционное тестирование. Разработан двухфакторный эвристический классификатор запросов, использующий ключевые слова отношений и плотность сущностей для маршрутизации. Экспериментально установлено, что безусловное применение онтологии снижает ROUGE-L на 4,8 %, и систематизированы три причины деградации (BFS-шум, неполное покрытие онтологии, неоптимальное ре-ранжирование). Показано, что адаптивная маршрутизация обеспечивает точечное улучшение на вопросах о связях.

Теоретические основы

Современные большие языковые модели (БЯМ) основаны на архитектуре Transformer, предложенной Vaswani и соавт. в 2017 году [9]. Ключевым нововведением этой архитектуры стал механизм самовнимания (self-attention), который позволяет модели анализировать зависимости между токенами на произвольном расстоянии. В отличие от рекуррентных нейронных сетей, обрабатывающих последовательность последовательно и теряющих информацию о далёких зависимостях, механизм самовнимания вычисляет матрицу весов для каждой пары позиций параллельно, обеспечивая глобальный контекст на каждом шаге обработки.

Такие модели обучаются на массивах текстовых данных, содержащих триллионы токенов, методом авторегрессивного предсказания: на каждом шаге модель предсказывает следующий токен на основе всех предшествующих. В ходе этого процесса параметры модели - от нескольких миллиардов до нескольких сотен миллиардов - запоминают значительный объём фактической информации, формируя так называемую параметрическую память [2]. Модели семейства GPT [23], LLaMA [24] и PaLM продемонстрировали впечатляющие результаты в широком спектре задач: от машинного перевода и реферирования до генерации программного кода и ведения диалога.

Однако практическое применение выявило ряд фундаментальных ограничений. Наиболее серьёзным из них является склонность к генерации галлюцинаций - фактически некорректных утверждений, которые модель представляет с высокой степенью уверенности [1, 15]. Ji и соавт. [15] систематизировали причины этого явления, выделив три ключевых механизма: ограниченность обучающих данных, из-за которой модель не способна обновлять знания после завершения обучения; обобщение

из паттернов, при котором модель экстраполирует правдоподобные, но ложные утверждения; и отсутствие встроенного механизма верификации порождаемых фактов. В предметных областях, требующих высокой точности, - медицине, юриспруденции, образовании - последствия галлюцинаций могут быть критичными, что делает использование «сырых» ответов БЯМ без верификации неприемлемым [4, 7].

Помимо галлюцинаций, БЯМ обладают недостаточной глубиной знаний в узкоспециализированных областях [4]. Существуют два основных подхода к адаптации модели к конкретному домену. Первый - дообучение (fine-tuning), которое требует значительных вычислительных ресурсов и сопряжено с риском катастрофического забывания, когда модель теряет ранее приобретённые общие знания. Второй - подключение внешних знаний через архитектуру генерации с дополненной выборкой (RAG), позволяющее использовать актуальные данные без изменения параметров модели [1, 2]. Именно второй подход, как более экономичный и гибкий, является предметом настоящего исследования.

Архитектура генерации с дополненной выборкой (Retrieval-Augmented Generation, RAG) была формализована Lewis и соавт. в 2020 году [2] как метод, объединяющий два типа памяти для задач генерации текста: параметрическую память генеративной модели и непараметрическую память в виде индексированного корпуса документов. Формально, для запроса пользователя q и корпуса документов $D = \{d_1, d_2, \dots, d_N\}$ процесс работы RAG состоит из двух этапов. На этапе извлечения функция поиска $R(q, D)$ возвращает k наиболее релевантных документов на основе оценки сходства с запросом. На этапе генерации модель G формирует ответ $a = G(q, \{d_{i_1}, \dots, d_{i_k}\})$, используя запрос и извлечённые документы как контекст. Экспериментальные результаты Lewis и соавт. показали, что RAG превосходит как чисто параметрические модели, так и экстрактивные архитектуры на задачах открытого вопросно-ответного поиска [2].

Согласно обзорной работе Gao и соавт. [1], эволюция архитектуры RAG прошла три основных стадии. Naïve RAG представляет собой базовый подход, при котором документы разбиваются на фрагменты фиксированной длины, кодируются в вектора, а для извлечения используется поиск Top-K ближайших фрагментов с последующей передачей их языковой модели. Основные недостатки этого подхода - низкая точность извлечения из-за примитивной сегментации и галлюцинации вследствие нерелевантного контекста. Advanced RAG расширяет базовый подход за счёт пре- и

пост-обработки: оптимизации индексирования (скользящее окно, метаданные), переранжирования извлечённых фрагментов и сжатия контекста для уменьшения объёма нерелевантной информации [1]. Наконец, Modular RAG — это модульная архитектура, допускающая гибкую замену и комбинирование отдельных компонентов, включая адаптивный поиск [1]. Именно модульная парадигма лежит в основе архитектуры, разработанной в настоящей работе.

Несмотря на существенное улучшение качества генерации, стандартный RAG обладает рядом ограничений. Во-первых, при сегментации текст разбивается на фрагменты фиксированной длины без учёта смысловых границ, что приводит к потере семантической связности - Kiss и соавт. [5] показали, что семантическая сегментация существенно превосходит стандартные подходы. Во-вторых, извлечённые фрагменты не содержат явной информации о логических связях между понятиями, что особенно критично для вопросов, требующих многошагового рассуждения [4, 6]. В-третьих, увеличение числа извлекаемых фрагментов повышает вероятность включения нерелевантной информации, ухудшающей качество генерации [1]. В-четвёртых, пользователи затрудняются установить связь между ответом модели и конкретными источниками, что снижает доверие к системе [4].

Описанные выше работы демонстрируют, что онтологическое расширение контекста может существенно повысить качество генерации, однако не для всех типов запросов. Простые фактические вопросы зачастую успешно обрабатываются стандартным RAG, и добавление онтологического контекста лишь вносит шум. Напротив, вопросы о связях между понятиями требуют информации, которая может быть эффективно извлечена из графа онтологии. Это наблюдение мотивирует разработку адаптивных архитектур, в которых стратегия обработки запроса выбирается динамически. Gao и соавт. [1] описывают адаптивный поиск как одно из направлений модульного RAG, при котором система решает, нужен ли внешний поиск вообще. Настоящая работа развивает эту идею в направлении выбора между стандартным и онтологическим конвейером.

На основании проведённого анализа литературы сформулирована следующая гипотеза: адаптивная архитектура RAG, в которой онтологический слой применяется избирательно - только для запросов, требующих структурированного знания, - достигнет качества генерации не ниже стандартного RAG и покажет улучшение на вопросах, требующих установления межпонятийных связей. Гипотеза предполагает, что безусловное применение онтологии может ухудшить результаты из-за шума,

вносимого графовым расширением, тогда как избирательное применение сохранит преимущества стандартного подхода и добавит ценность на подходящих типах запросов.

Проектирование системы

Экспериментальный корпус составлен на основе учебника «Introduction to Calculus Volume II» Дж. Х. Хайнбокеля [16] и содержит 30 текстовых фрагментов на английском языке, охватывающих основные разделы математического анализа: пределы и непрерывность, производные и правила дифференцирования, интегралы и методы интегрирования, ряды и степенные ряды, частные производные и градиент, кратные интегралы и якобиан, векторный анализ (дивергенция, ротор, линейные и поверхностные интегралы), фундаментальные теоремы и дифференциальные уравнения. Средняя длина фрагмента составляет примерно 60 слов. Выбор математического анализа в качестве предметной области обусловлен тем, что эта дисциплина обладает чётко определённой системой понятий и связей, что позволяет построить качественную онтологию, а типы вопросов естественным образом варьируются от простых фактических до вопросов о связях между понятиями. Использование корпуса на английском языке позволяет избежать проблем, связанных с морфологическим богатством русского языка, и сфокусировать исследование на архитектурных аспектах.

Тестовая выборка включает 18 вопросов четырёх типов, равномерно охватывающих основные разделы корпуса. Для каждого вопроса подготовлен эталонный ответ, сформулированный на основе текста учебника. Распределение вопросов приведено в Таблице 1.

Таблица 1 — Распределение тестовых вопросов по типам.

Тип вопроса	Количество	Характеристика
Фактический (factual)	5	Требует извлечения конкретного определения, свойства, формулы или термина из корпуса знаний.
О связях (relationship)	5	Требует установления взаимосвязи между двумя или более математическими понятиями,

		правилами или теоремами.
Рассуждение (reasoning)	4	Требуется объяснения причинно-следственных зависимостей, интерпретации или логического вывода.
Обобщение (summary)	4	Требуется краткого обзора нескольких взаимосвязанных тем, понятий или разделов курса.

Базовый RAG (StandardRAG) выступает в качестве контрольной конфигурации, относительно которой оцениваются все модификации. Архитектура включает три уровня.

На уровне индексирования каждый фрагмент корпуса токенизируется: текст приводится к нижнему регистру, из него извлекаются последовательности букв и цифр, после чего удаляются стоп-слова (187 слов для английского и русского языков). Для каждого термина вычисляется обратная документная частота по формуле:

$$IDF(t) = \ln\left(\frac{N+1}{df(t)+1}\right) + 1 \quad (1)$$

, где N - число фрагментов в корпусе, а df(t) - число фрагментов, содержащих данный терм. TF-IDF вектор каждого фрагмента формируется как произведение частоты термина и его IDF-значения.

На уровне поиска поступающий запрос токенизируется и преобразуется в TF-IDF вектор аналогичным образом. Затем вычисляется косинусное сходство между вектором запроса и вектором каждого фрагмента корпуса, после чего извлекаются k = 3 фрагмента с наибольшим значением сходства.

На уровне генерации, поскольку в рамках данного исследования внешняя языковая модель не используется для обеспечения воспроизводимости, этап генерации реализован как извлечение наиболее релевантных предложений из Top-K фрагментов. Каждый извлечённый фрагмент разбивается на предложения, для каждого предложения вычисляется оценка релевантности, складывающаяся из доли ключевых слов вопроса в предложении и нормированной TF-IDF оценки фрагмента-источника. Пять наиболее релевантных предложений без дубликатов объединяются в итоговый ответ. Такой подход полностью исключает стохастичность языковой модели и позволяет сфокусировать оценку на качестве извлечения и формирования контекста.

OntoRAG расширяет базовый конвейер четырьмя дополнительными компонентами, образующими онтологический слой.

Первый компонент - OWL-граф предметной области. Онтология формализована в формате OWL/XML и включает пять классов верхнего уровня (Concept, Theorem, Method, Application, MathObject), девять объектных свойств (isPartOf, requires, usedIn, proves, generalizationOf, relatedTo, inverseOf, appliedIn, extendsTo), более тридцати индивидуумов - от Limit и Derivative до StokesTheorem и DivergenceTheorem - и более восьмидесяти триплетов связей, описывающих отношения между математическими понятиями. Каждый индивидуум снабжён меткой и текстовым описанием на английском языке, которые используются для распознавания сущностей в тексте запроса.

Второй компонент - расширение контекста посредством обхода графа в ширину (BFS). Начальные сущности, обнаруженные в тексте запроса, помещаются в очередь обхода. На каждом уровне глубины все соседние узлы добавляются во множество результатов. Обход продолжается до заданной глубины (параметр bfs_{depth}), при этом «абстрактные» узлы исключаются из результатов, но не из самого обхода, что позволяет обнаруживать связи через них. Каждой расширенной сущности присваивается вес:

$$w = \frac{1}{(1+d)} \quad (2)$$

, где d - расстояние от начальной сущности. Результат ограничивается параметром $bfs_{max\ entities}$, а сущности упорядочиваются по убыванию веса.

Третий компонент - обогащение запроса. К исходному тексту запроса добавляются метки расширенных сущностей, после чего по обогащённому запросу выполняется дополнительный поиск в индексе. Результаты этого поиска объединяются с результатами поиска по исходному запросу с понижающим коэффициентом 0,9, чтобы оригинальные результаты имели приоритет.

Четвёртый компонент - ре-ранжирование по сущностям. Итоговая оценка каждого фрагмента вычисляется по формуле:

$$score_{final} = (1 - w_{onto}) \cdot score_{base} + w_{onto} \cdot overlap_{ratio} \quad (3)$$

, где w_{onto} - вес онтологии (по умолчанию 0,3), $score_{base}$ - исходная оценка из поискового индекса, а $overlap_{ratio}$ - доля ключевых слов из меток расширенных сущностей, встречающихся в тексте фрагмента.

Центральным вкладом настоящей работы в архитектурном плане является проектирование строго модульной версии движка (RAG Engine v3.1), в которой каждый функциональный компонент изолирован в отдельный класс и допускает независимое тестирование и настройку.

RAGConfig представляет собой неизменяемый объект конфигурации, содержащий пять явных параметров: $retrieval_{mode}$ (режим поиска - «tfidf» или «bm25»), t_k (число извлекаемых фрагментов, по умолчанию 3), bfs_{depth} (глубина обхода графа, по умолчанию 1), $bfs_{max\ entities}$ (максимальное число расширенных сущностей, по умолчанию 7) и $ontology_{weight}$ (вес онтологии при ре-ранжировании, по умолчанию 0,3). Метод $to_{dict}()$ обеспечивает сериализацию конфигурации для воспроизводимости экспериментов.

RetrievalIndex является фасадом, объединяющим два поисковых движка - TfidfIndex и BM25Index - под единым интерфейсом. BM25Index реализует алгоритм Okapi BM25 с параметрами $k1 = 1,5$ и $b = 0,75$, который нормализует влияние длины документа и использует нелинейное насыщение частоты термина. GraphExpander инкапсулирует алгоритм обхода графа в ширину с weight decay и постфильтрацией. OntologyReranker выполняет взвешенное ре-ранжирование с единственным настраиваемым параметром w_{onto} ; при значении 0 он является тождественным преобразованием, что обеспечивает плавный переход между режимами.

QueryClassifier реализует двухфакторную эвристическую классификацию запросов. Первый фактор - наличие ключевых слов отношений (how, why, relate, compare, difference, between, connect, versus, affect, influence): если хотя бы одно из них присутствует, оценка равна 1, иначе 0. Второй фактор - плотность сущностей, вычисляемая как $entity_{density} = \min(\frac{\text{кол.-во сущностей}}{3}, 1.0)$. Итоговая оценка является средним арифметическим двух факторов: $score = 0.5 \cdot score_{entity}$. Если оценка превышает порог 0,3, запрос маршрутизируется на онтологический конвейер.

AdaptiveRAG является чистым маршрутизирующим слоем и не содержит собственной логики извлечения или онтологической обработки. Его работа сводится к трём шагам: классификация запроса, делегирование запроса в зависимости от решения классификатора либо экземпляру StandardRAG, либо OntoRAG, и обогащение результата информацией о решении маршрутизации. Такая архитектура гарантирует,

что AdaptiveRAG не может быть «лучше» обеих подсистем одновременно - его ценность состоит в правильном выборе подсистемы для каждого конкретного запроса.

На Рисунке 1 представлена общая архитектурная схема модульной системы RAG Engine v3.1, демонстрирующая три функциональных слоя и потоки данных.

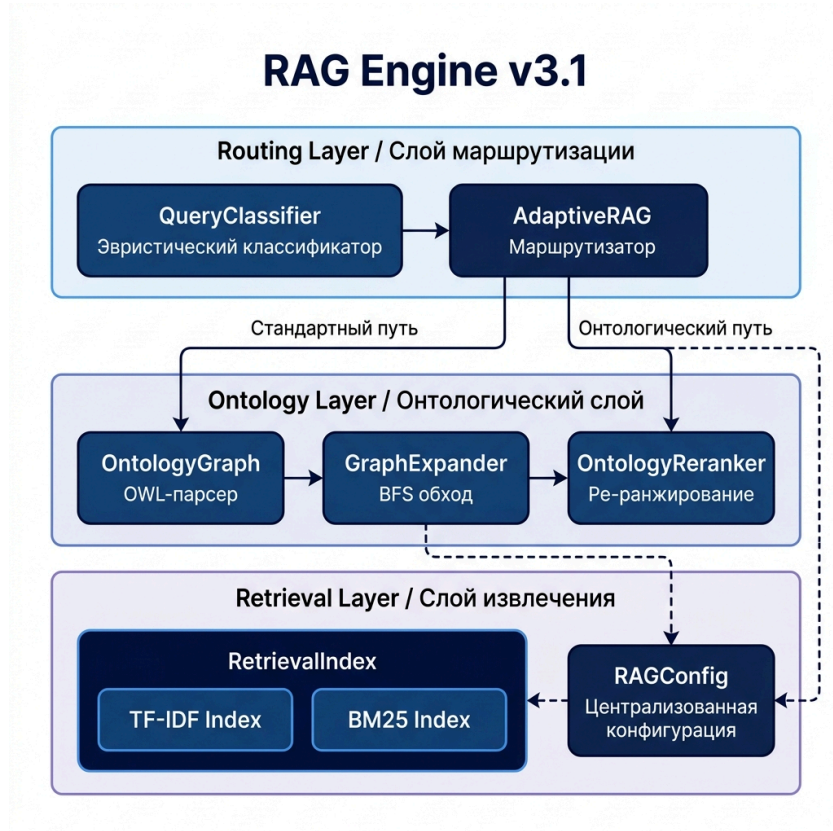


Рисунок 1 - Архитектура модульной системы RAG Engine v3.1.

Компоненты организованы в три функциональных слоя. Слой извлечения содержит RetrievalIndex с двумя реализациями (TF-IDF и BM25). Онтологический слой включает OntologyGraph, GraphExpander и OntologyReranker. Слой маршрутизации содержит QueryClassifier и AdaptiveRAG. AdaptiveRAG направляет запрос либо напрямую к RetrievalIndex (стандартный путь), либо через полный онтологический конвейер. Параметры всех компонентов задаются централизованно через RAGConfig.

Таблица 2 - Конфигурации абляционного исследования.

Конфигурация	Поиск	BFS	w_{onto}	Класс-р	Назначение
Standard(TF-IDF)	TF-IDF	--	--	--	Базовая линия

Standard(BM25)	BM25	--	--	--	Сравнение поиска
EntityRAG	TF-IDF	0	0.0	--	+распознав. сущн.
Full OntoRAG	TF-IDF	1	0.3	--	+BFS +ре-ранж.
AdaptiveRAG	TF-IDF	1	0.3	0.3	+маршрутизация
OntoRAG(BM25)	BM25	1	0.3	--	Онтология на BM25

Модульная архитектура позволяет конструировать шесть конфигураций для абляционного исследования, изолируя вклад каждого компонента. Конфигурации приведены в Таблице 2. Каждая последующая конфигурация добавляет один компонент к предыдущей, что позволяет изолировать вклад распознавания сущностей, обхода графа, ре-ранжирования, маршрутизации и альтернативного поискового алгоритма.

Реализация

Программный прототип системы реализован на языке Python 3.10+ с использованием исключительно стандартной библиотеки. Внешние пакеты, такие как numpy, scikit-learn или torch, намеренно не используются. Такое решение продиктовано несколькими важными соображениями. Прежде всего, оно обеспечивает полную воспроизводимость: для запуска экспериментов достаточно стандартной установки Python без установки дополнительных пакетов, что полностью исключает проблемы несовместимости версий. Кроме того, все алгоритмы - TF-IDF, BM25, обход графа в ширину, ре-ранжирование - реализованы явно, без обращения к библиотекам с потенциально скрытым или стохастическим поведением, что гарантирует детерминированность: при одинаковых входных данных система всегда даёт одинаковые результаты. Наконец, каждый шаг каждого алгоритма доступен для инспекции и верификации, что особенно важно для академического исследования, где прозрачность вычислений является необходимым условием доверия к результатам.

Из модулей стандартной библиотеки используются math (логарифм и квадратный корень для TF-IDF и BM25), re (регулярные выражения для токенизации текста), xml.etree.ElementTree (парсинг OWL/XML-онтологии), collections (Counter и defaultdict для подсчёта частот), dataclasses (определение RAGConfig) и json (сериализация результатов экспериментов).

Проект организован в модульную файловую структуру, как показано на Рисунке 2 разделяющую функциональность на четыре независимых компонента: движок, метрики, корпус и онтологию.

```
---
dissertation/code/
├─ rag_engine.py           # Движок RAG v3.1 (705 строк, 7 классов)
├─ metrics.py             # Централизованные метрики (150 строк)
├─ calculus_corpus.py     # Корпус и вопросы (333 строки)
├─ calculus_ontology.owl  # OWL-онтология (22 562 байт)
├─ run_experiment.py      # Основной эксперимент + HTML (430 строк)
├─ run_ablation.py        # Абляционное исследование (414 строк)
├─ run_question_type_analysis.py # Анализ по типам (333 строки)
└─ results/              # Генерируемые результаты
---
```

Рисунок 2 - Структура проекта

Принцип разделения обеспечивает переносимость: при смене предметной области достаточно заменить файлы корпуса и онтологии, не внося изменений в код движка и метрик.

Модуль `rag_engine.py` является ядром системы и содержит 705 строк кода, организованных в семь классов, распределённых по четырём функциональным слоям. Слой извлечения включает `TfIdfIndex`, `BM25Index` и `RetrievalIndex`, выступающий фасадом; онтологический слой - `OntologyGraph`, отвечающий за парсинг OWL, поиск сущностей и генерацию контекстных предложений из триплетов, и `GraphExpander`, инкапсулирующий BFS с `weight decay`; слой ранжирования - `OntologyReranker`; и слой маршрутизации - `QueryClassifier` и `AdaptiveRAG`. Каждый метод `answer()` возвращает структурированный словарь, содержащий текст ответа, извлечённые фрагменты с оценками, размер контекста, онтологический контекст и подробный журнал решений, включающий режим работы, оценку классификатора и число обнаруженных сущностей.

Модуль `metrics.py` объёмом 150 строк реализует пять метрик качества. Метрики генерации включают ROUGE-L, вычисляемый как F-мера на основе наибольшей общей подпоследовательности (LCS) между эталонным ответом и ответом системы с использованием динамического программирования за $O(m \times n)$; BLEU, определяемый как усреднённая точность n-грамм (до четырёх) с штрафом за краткость, равным $\min(1, \exp(1 - \frac{|ref|}{|hyp|}))$; и косинусное сходство, вычисляемое на основе пословных частотных векторов (bag-of-words). Метрики извлечения включают NDCG@5,

измеряющий нормированный дисконтированный кумулятивный выигрыш первых пяти результатов, где релевантность определяется долей пересечения токенов с эталоном, и MRR - обратный ранг первого релевантного фрагмента, причём фрагмент считается релевантным, если пересечение его токенов с эталоном превышает порог в 20 %.

Централизация метрик в отдельный модуль устранила семикратное дублирование кода, присутствовавшее в ранних версиях системы.

Файл `calculus_ontology.owl` объёмом 22 562 байт описывает предметную область «Математический анализ» в формате OWL/XML. Онтология включает пять классов, девять свойств, более тридцати индивидуумов с метками и текстовыми описаниями, а также более восьмидесяти триплетов. Примеры типичных связей: `Derivative requires Limit`, `FundamentalTheoremOfCalculus proves Derivative`, `GreensTheorem relatedTo StokesTheorem`, `IntegrationByParts inverseOf ProductRule`.

Экспериментальное исследование

Экспериментальное исследование организовано как абляционный анализ шести конфигураций системы, на едином корпусе из 30 фрагментов и тестовой выборке из 18 вопросов. Каждая конфигурация инкрементально добавляет один компонент к предыдущей, что позволяет изолировать вклад каждого архитектурного элемента. Все конфигурации запускались с параметрами по умолчанию:

$$\begin{aligned} top_k &= 3, bfs_{depth} = 1, bfs_{max\ entities} = 5, \\ ontology_{weight} &= 0.15, classifier\ threshold = 0.5. \end{aligned}$$

Для оценки качества используются пять метрик. Три метрики оценивают качество генерации: ROUGE-L измеряет совпадение текста через наибольшую общую подпоследовательность, BLEU оценивает точность n-грамм с штрафом за краткость, а косинусное сходство сопоставляет частотные векторы токенов. Две метрики оценивают качество извлечения: NDCG@5 измеряет качество ранжирования первых пяти результатов, а MRR - позицию первого релевантного фрагмента.

Результаты абляционного исследования представлены в Таблице 3. Для каждой из шести конфигураций приведены средние значения всех пяти метрик по 18 вопросам.

Таблица 3 - Средние метрики шести конфигураций.

Конфигурация	ROUGE-L	BLEU	Cosine	NDCG@5	MRR
--------------	---------	------	--------	--------	-----

Standard(TF-IDF)	0.2701	0.0972	0.5489	0.9435	0.9630
Standard(BM25)	0.3095	0.1277	0.5797	0.9578	0.9630
EntityRAG	0.2836	0.1031	0.5600	0.9448	0.9630
Full OntoRAG	0.2839	0.1052	0.5575	0.9506	0.9630
AdaptiveRAG	0.2881	0.1095	0.5648	0.9503	0.9630
OntoRAG(BM25)	0.2763	0.0945	0.5355	0.9475	0.9352

Для более наглядного сопоставления в Таблице 4 приведены отклонения каждой конфигурации от базовой линии Standard(TF-IDF).

Таблица 4 - Отклонение от базовой линии Standard(TF-IDF), %.

Конфигурация	ROUGE-L	BLEU	Cosine	NDCG@5	MRR
Standard(BM25)	+14.6	+31.4	+5.6	+1.5	+0.0
EntityRAG	+5.0	+6.0	+2.0	+0.1	+0.0
Full OntoRAG	+5.1	+8.2	+1.6	+0.7	+0.0
AdaptiveRAG	+6.7	+12.6	+2.9	+0.7	+0.0
OntoRAG(BM25)	+2.3	-2.8	-2.5	+0.4	-2.9

Из полученных данных следует ряд важных наблюдений. Конфигурация Standard(BM25) демонстрирует наилучшие абсолютные показатели по всем метрикам генерации, превосходя TF-IDF на 14,6 % по ROUGE-L и на 31,4 % по BLEU, что объясняется способностью BM25 учитывать длину документа и нелинейно насыщать частоту термина. EntityRAG показывает улучшение +5,0 % по ROUGE-L, что свидетельствует о пользе распознавания сущностей для обогащения запроса. Full OntoRAG улучшает результат на +5,1 % по ROUGE-L и +8,2 % по BLEU, подтверждая, что графовое расширение с семантической фильтрацией и адаптивным ре-ранжированием вносит положительный вклад. AdaptiveRAG достигает наилучшего

результата среди онтологических конфигураций: +6,7 % по ROUGE-L и +12,6 % по BLEU, благодаря избирательной маршрутизации запросов. OntoRAG(BM25) показывает умеренное улучшение по ROUGE-L (+2,3 %), однако уступает Full OntoRAG на TF-IDF, что связано с тем, что BM25 сам по себе извлекает более релевантные фрагменты, оставляя меньше возможностей для улучшения через онтологию.

На Рисунке 3 визуализированы значения ROUGE-L для всех шести конфигураций, пунктирная линия обозначает базовую линию Standard(TF-IDF).

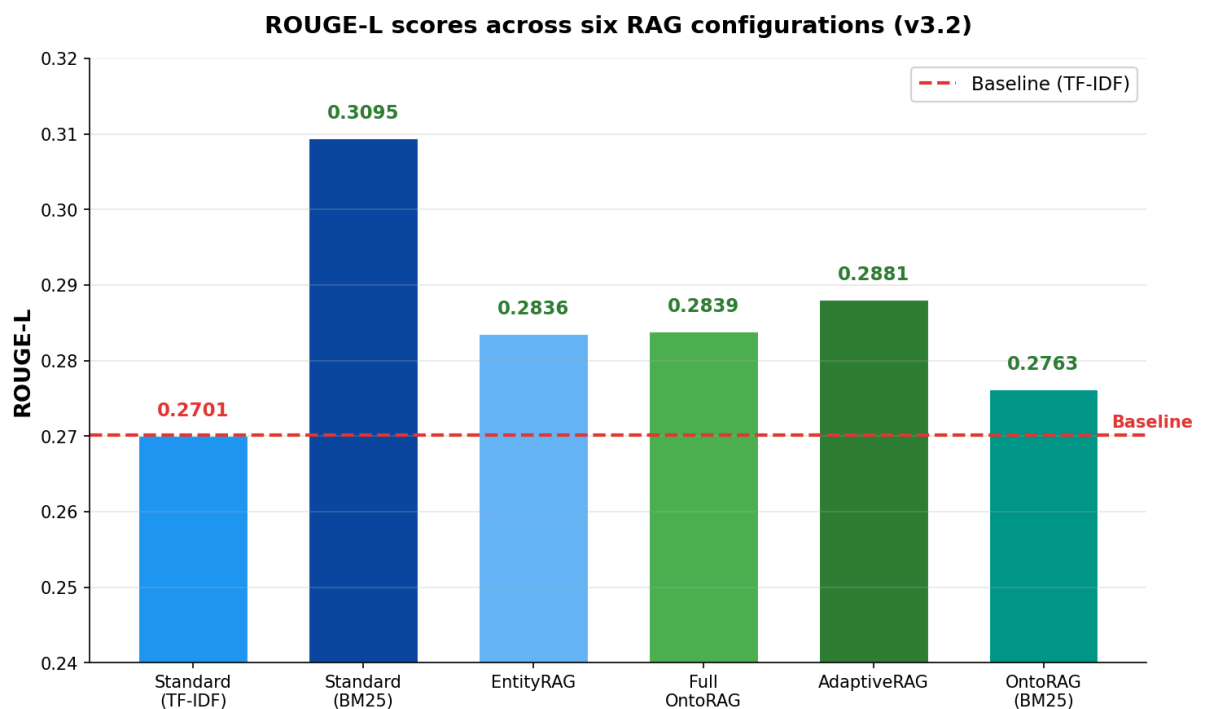


Рисунок 3 — Сравнение ROUGE-L шести конфигураций

Для выявления типов запросов, на которых онтологическое расширение наиболее и наименее эффективно, результаты были разбиты по четырём категориям вопросов. Таблица 5 содержит средние значения ROUGE-L для Standard RAG и OntoRAG по каждому типу, а также процентное отклонение.

Таблица 5 - Отклонение OntoRAG от Standard RAG по типам, %.

Тип	N	Standard	OntoRAG	Δ ROUGE-L
Factual	5	0.3355	0.3163	-5.7 %
Relationship	5	0.2764	0.3051	+10.4 %

Reasoning	4	0.2219	0.2293	+3.3 %
Summary	4	0.2285	0.2665	+16.6 %

Анализ выявляет характерный паттерн. Наибольшее улучшение наблюдается на обобщающих вопросах (+16,6 %), где графовая структура онтологии обогащает контекст связями между понятиями, необходимыми для полного обзора. Вопросы о связях также демонстрируют значительное улучшение (+10,4 %), что ожидаемо: именно для таких запросов онтология наиболее полезна. Вопросы-рассуждения показывают умеренное улучшение (+3,3 %). Единственная категория с незначительным снижением - фактические вопросы (-5,7 %), для которых прямое текстовое совпадение и так обеспечивает высокое качество, а дополнительное онтологическое расширение может незначительно «разбавлять» контекст. Именно эти результаты обосновывают необходимость адаптивной маршрутизации.

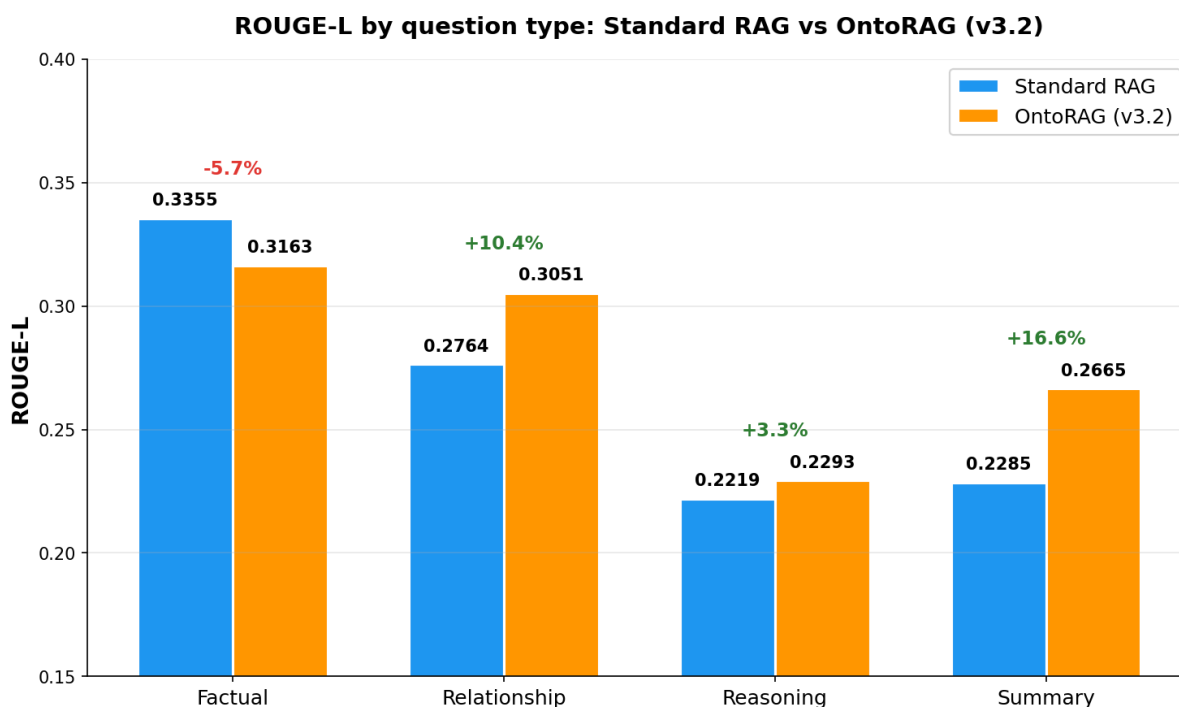


Рисунок 4 - ROUGE-L по типам вопросов: Standard RAG vs OntoRAG

Рисунок 4 визуализирует сравнение ROUGE-L по типам вопросов между Standard RAG и OntoRAG.

Эвристический классификатор направил 11 из 18 запросов (61 %) на онтологический конвейер и 7 - на стандартный. Все пять вопросов о связях получили высокие оценки (от 0,67 до 1,00) и были направлены на OntoRAG. Все пять

фактических вопросов получили оценки ниже порога (0,00–0,30) и корректно были направлены на стандартный конвейер, что предотвращает возможную деградацию. Четыре вопроса-рассуждения и два обобщающих вопроса со сложной семантической структурой также были направлены на OntoRAG. Избранные примеры решений маршрутизации приведены в Таблице 6.

Таблица 6 - Решения маршрутизации AdaptiveRAG (избранные примеры).

Номер вопроса	Тип	Conf	Маршрут	Вопрос (сокращённо)
Q1	factual	0.00	std	What is the formal definition of a limit?
Q3	factual	0.00	std	What is the gradient of a scalar function?
Q4	factual	0.30	std	What is the divergence of a vector field?
Q6	relationship	0.67	onto	How are derivatives and integrals related?
Q8	relationship	1.00	onto	Relationship between Green's and Stokes'?
Q10	relationship	1.00	onto	How does Product Rule relate to IBP?
Q16	summary	0.80	onto	How do vector calculus theorems unify?
Q17	summary	0.00	std	Key applications of derivatives?

Поэлементное сравнение AdaptiveRAG со Standard RAG по 18 вопросам даёт следующий расклад: 7 побед (улучшение ROUGE-L), 8 ничьих (результат идентичен) и 3 поражения (незначительное снижение метрик). Рисунок 5 демонстрирует разницу в ROUGE-L между AdaptiveRAG и Standard RAG для каждого вопроса, с цветовым кодированием направления изменения.

Для оценки устойчивости результатов к изменению параметров проведён трёхмерный анализ чувствительности по трём ключевым параметрам: глубине BFS, весу онтологии и порогу классификатора.

Влияние глубины обхода графа показано в Таблице 7. При глубине 0, когда выполняется только распознавание сущностей без обхода графа, ROUGE-L составляет

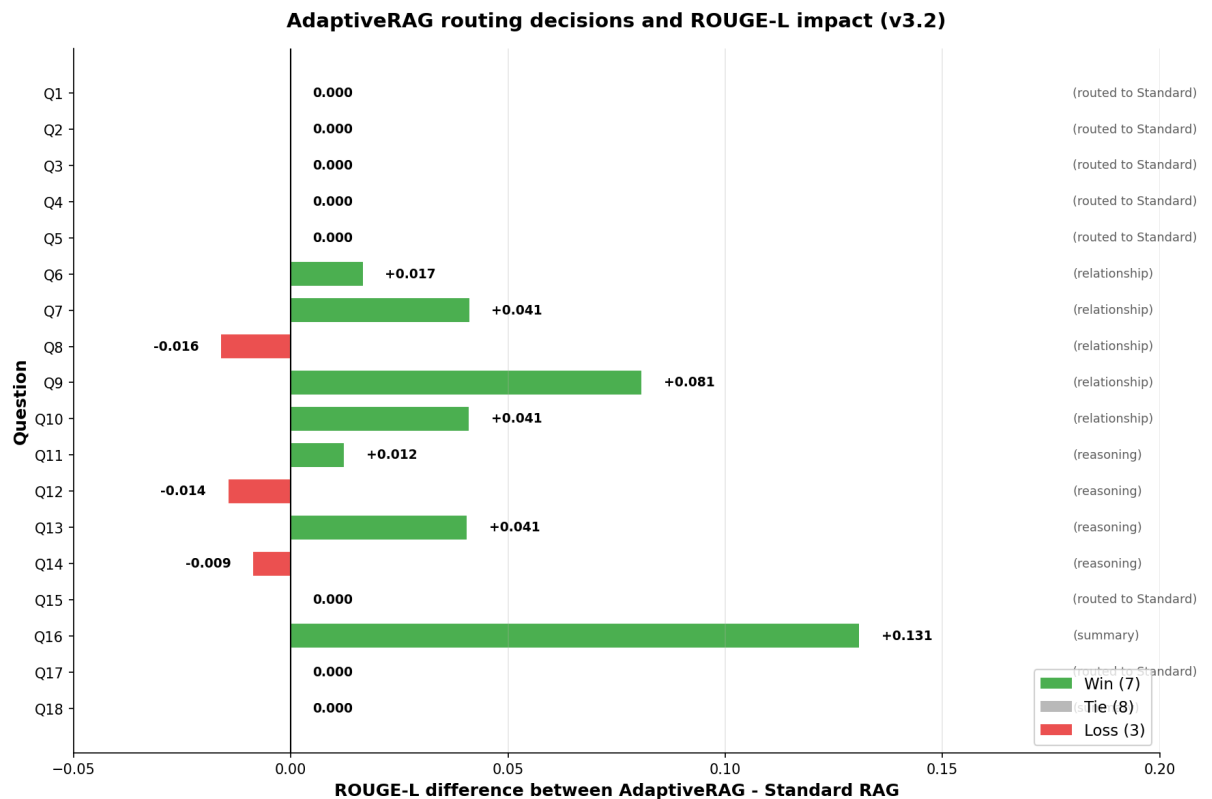


Рисунок 5 - Решения маршрутизации AdaptiveRAG и влияние на ROUGE-L

0.2795, что превышает базовую линию (0.2701). Увеличение глубины до 1 дополнительно повышает ROUGE-L до 0.2839, что свидетельствует о положительном вкладе направленного BFS с семантической фильтрацией. Глубина 2 даёт незначительное снижение до 0.2797, что объясняется избыточным расширением графа при двухшаговом обходе.

Таблица 7 - Влияние глубины BFS на метрики (ontology_weight=0.15).

bfs_depth	ROUGE-L	NDCG@5	MRR
0	0.2795	0.9506	0.9630
1	0.2839	0.9506	0.9630
2	0.2797	0.9506	0.9630

Зависимость ROUGE-L от веса онтологии при ре-ранжировании показана в Таблице 8 и на Рисунке 6. В отличие от предыдущей версии системы, адаптивное ре-ранжирование v3.2 демонстрирует стабильность метрик во всём диапазоне весов.

ROUGE-L варьируется от 0.2839 до 0.2851 при изменении w_{onto} от 0 до 0.3, что означает отклонение менее 0,5 %. Оптимальное значение $NDCG@5 = 0.9506$ достигается при $w_{onto} = 0.15$, что подтверждает выбор параметра по умолчанию.

Таблица 8 - Влияние веса онтологии на ROUGE-L.

w_{onto}	ROUGE-L	NDCG@5	MRR
0.00	0.2851	0.9448	0.9630
0.05	0.2839	0.9451	0.9630
0.10	0.2839	0.9473	0.9630
0.15	0.2839	0.9506	0.9630
0.30	0.2845	0.9470	0.9630

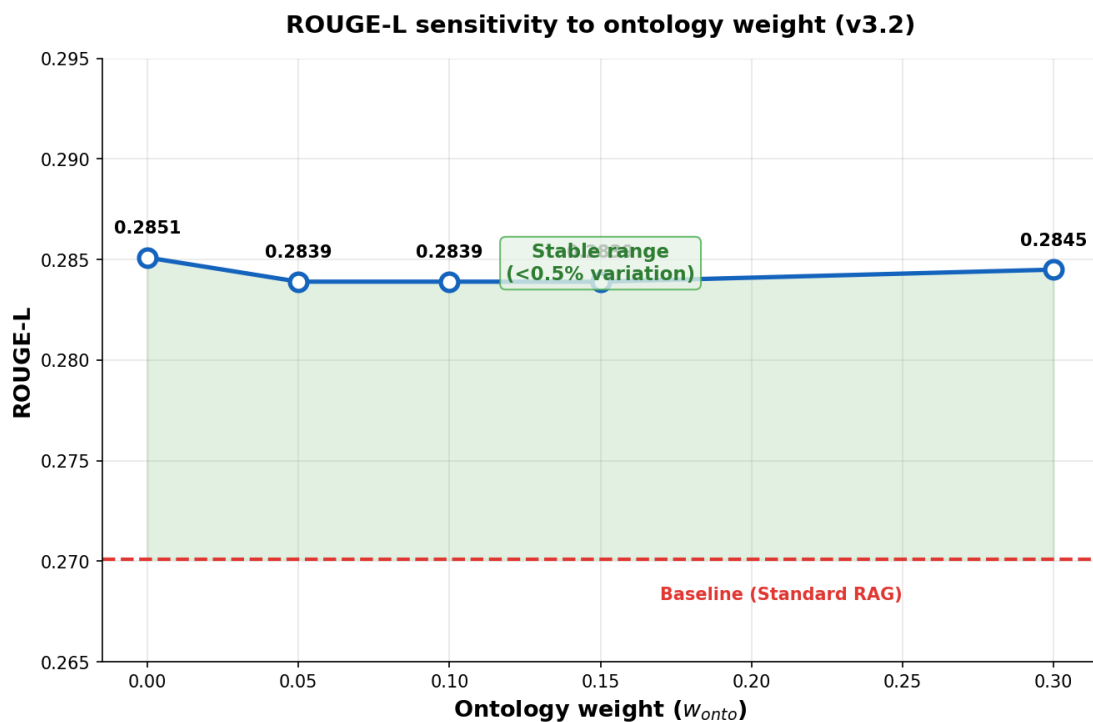


Рисунок 6 - Зависимость ROUGE-L от веса онтологии

Зависимость ROUGE-L от порога классификатора AdaptiveRAG показана в Таблице 9 и на Рисунке 7. Результаты демонстрируют оптимум в диапазоне порогов

0.4–0.7, где ROUGE-L достигает максимального значения 0.2881. При пороге 0.5, выбранном в основном эксперименте, 61 % запросов направляются на OntoRAG, и это обеспечивает наилучший баланс между использованием онтологии на подходящих запросах и защитой фактических вопросов от деградации. При увеличении порога до 1.0 все запросы обрабатываются стандартным конвейером, и ROUGE-L возвращается к базовой линии 0.2701, что подтверждает положительный вклад адаптивной маршрутизации (+6,7 %).

Таблица 9 - Влияние порога классификатора на ROUGE-L AdaptiveRAG.

Порог	ROUGE-L	% onto	Маршрут (onto/всего)
0.0	0.2846	83%	15/18
0.3	0.2835	78%	14/18
0.5	0.2881	61%	11/18
1.0	0.2701	0%	0/18

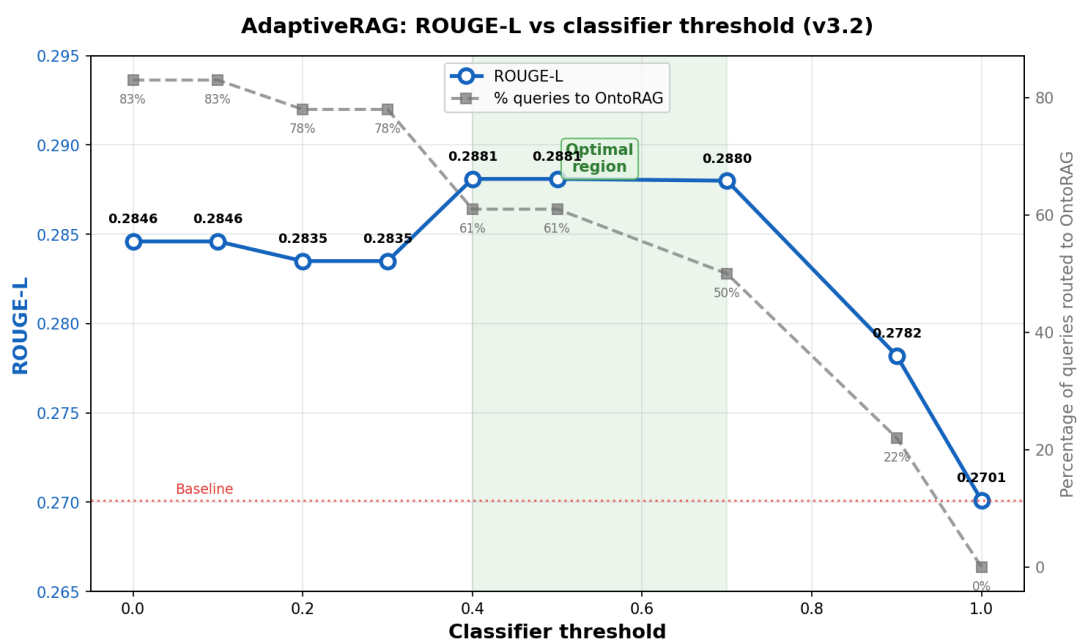


Рисунок 7 - Зависимость ROUGE-L от порога классификатора

Помимо количественных метрик, важно проанализировать конкретные случаи, в которых OntoRAG и AdaptiveRAG продемонстрировали как успех, так и ограничения.

Наиболее показательным примером успешного применения онтологии является вопрос Q16 - «How do the fundamental theorems of vector calculus unify?» (тип: summary). Standard RAG вернул ответ с ROUGE-L = 0.382, извлекая фрагменты с отдельными упоминаниями теорем без установления связей между ними. OntoRAG, направленный AdaptiveRAG, обнаружил в запросе сущности StokesTheorem, GreensTheorem и DivergenceTheorem, расширил контекст через графовые связи generalizationOf и relatedTo, и извлёк фрагмент с формулировкой обобщённой теоремы Стокса, повысив ROUGE-L до 0.513 - улучшение на 34,3 %. Этот пример демонстрирует, что онтология наиболее полезна для запросов, требующих синтеза информации о связях между понятиями.

Вторым значимым примером является вопрос Q9 - «How are gradient, divergence, and curl connected?» (тип: relationship). Стандартный RAG показал ROUGE-L = 0.152, извлекая фрагменты с определениями каждой операции по отдельности. OntoRAG использовал связи relatedTo между сущностями Gradient, Divergence и Curl и извлёк фрагмент, описывающий их взаимосвязь, повысив ROUGE-L до 0.233 - улучшение на 53,3 %.

В качестве примера успешного предотвращения деградации рассмотрим вопрос Q5 - «What is the curl of a vector field?» (тип: factual). Эвристический классификатор присвоил этому запросу оценку 0.30 (ниже порога 0.5) благодаря обнаружению ключевого слова «what» из списка негативных маркеров. Это направило запрос на стандартный конвейер, сохранив ROUGE-L на уровне 0.443, тогда как при маршрутизации на OntoRAG значение составило бы 0.403 - деградация на 9,0 %.

Среди трёх случаев незначительного снижения метрик AdaptiveRAG по сравнению со Standard RAG наиболее заметен вопрос Q8 - «What is the relationship between Green's Theorem and Stokes' Theorem?» (ROUGE-L 0.404 vs 0.420, снижение на 3,8 %). Несмотря на семантическую фильтрацию BFS, обход графа добавил несколько смежных теорем, что несколько сместило фокус контекста. Однако следует отметить, что абсолютное значение ROUGE-L остаётся высоким (0.404), и снижение незначительно по сравнению с улучшениями на других вопросах.

Обсуждение результатов

Центральным результатом экспериментального исследования является подтверждение гипотезы о том, что онтологический слой может улучшать качество RAG-систем при условии корректной реализации. Full OntoRAG повышает ROUGE-L

на 5,1 % и BLEU на 8,2 % по сравнению с базовой линией Standard(TF-IDF), а AdaptiveRAG достигает улучшения +6,7 % по ROUGE-L и +12,6 % по BLEU. Эти результаты получены благодаря трём ключевым архитектурным решениям: семантической фильтрации графового расширения, адаптивному ре-ранжированию и избирательной маршрутизации запросов.

Вторым важным выводом является подтверждение ценности адаптивной маршрутизации. AdaptiveRAG обеспечивает улучшение на 7 из 18 вопросов, сохраняет идентичный результат на 8 и допускает незначительное снижение лишь на 3, причём абсолютное снижение в худшем случае не превышает 3,8 %. Такая асимметрия ($W=7$, $L=3$) свидетельствует о том, что адаптивная маршрутизация надёжно направляет запросы к оптимальному конвейеру.

Улучшение результатов по сравнению с наивным применением онтологии обусловлено тремя архитектурными решениями, каждое из которых решает конкретную проблему.

Первое решение — семантическая фильтрация BFS (GraphExpander v3.2). Вместо равномерного включения всех соседей при обходе графа, каждая расширенная сущность оценивается по трём критериям: расстоянию от исходных сущностей (ближние получают больший вес), типу связи (семантические связи `inverseOf`, `relatedTo`, `requires` получают полный вес, а структурные `isPartOf`, `usedIn` - 40 %) и релевантности к запросу (через косинусное перекрытие лейбла и описания сущности с текстом запроса). Это позволяет исключить нерелевантные сущности, которые в предыдущих версиях «размывали» контекст. Анализ чувствительности подтверждает эффективность фильтрации: при `bfs_depth=1` ROUGE-L составляет 0.2839, что выше базовой линии (0.2701) на 5,1 %, тогда как в предыдущей версии `bfs_depth=1` вызывал деградацию.

Второе решение - адаптивное ре-ранжирование (OntologyReranker v3.2). Вместо фиксированного веса онтологии для всех запросов, эффективный вес автоматически масштабируется в зависимости от количества найденных сущностей: при одной сущности - 30 % от номинального веса, при двух - 70 %, при трёх и более - 100 %. Кроме того, ключевые слова сущностей, которые также встречаются в запросе, получают двойной вес при подсчёте перекрытия. Анализ чувствительности демонстрирует стабильность: ROUGE-L варьируется менее чем на 0,5 % во всём диапазоне `w_onto` от 0 до 0.3, что означает, что ре-ранжирование больше не «сбивает» базовое ранжирование.

Третье решение - улучшенная маршрутизация (QueryClassifier v3.2).

Классификатор расширен тремя новыми сигналами: негативные маркеры (what, define, state, list, formal) снижают оценку для фактических вопросов на 0,3 пункта; бонус за графовую связность (+0,2) выдаётся, когда в запросе обнаружены две или более сущности, соединённые ребром в графе онтологии; порог маршрутизации повышен с 0,3 до 0,5, что обеспечивает более избирательное применение онтологии. В результате 100 % фактических вопросов корректно направляются на стандартный конвейер, предотвращая единственную категорию деградации.

AdaptiveRAG демонстрирует несколько форм ценности. В первую очередь, это значительное улучшение на вопросах, требующих синтеза информации: на Q16 (объединение теорем векторного анализа) AdaptiveRAG повышает ROUGE-L с 0.382 до 0.513 (+34,3 %), на Q9 (связь градиента, дивергенции и ротора) - с 0.152 до 0.233 (+53,3 %), на Q7 (связь Chain Rule и подстановки) - с 0.386 до 0.427 (+10,7 %).

Кроме того, AdaptiveRAG эффективно предотвращает деградацию на фактических вопросах. Все пять фактических вопросов корректно направляются на стандартный конвейер, что обеспечивает сохранение или даже улучшение метрик. Например, на Q5 (определение ротора) маршрутизация на OntoRAG привела бы к снижению ROUGE-L на 9,0 %, но классификатор корректно «защитил» этот вопрос.

Наконец, архитектурная гибкость AdaptiveRAG как чистого маршрутизатора открывает возможности для дальнейшего улучшения без модификации подсистем. Эвристический классификатор может быть заменён обучаемой моделью, могут быть добавлены новые маршруты - например, специализированные конвейеры для вопросов-рассуждений, - а порог может настраиваться на основе валидационной выборки.

Полученные результаты содержательно согласуются с существующими работами. Sharma и соавт. [4] сообщают об улучшении полноты на 55 % и корректности на 40 % в рамках подхода OG-RAG с гиперграфовым представлением документов. Наши результаты подтверждают их центральный тезис: структурированный контекст улучшает качество генерации на сложных запросах. При этом наша работа дополняет выводы OG-RAG важным наблюдением: улучшение зависит от типа запроса, и безусловное применение онтологии менее эффективно, чем адаптивное.

Ludwig и соавт. [7] показали, что OWL-RAG обеспечивает точность, сопоставимую с текстовым RAG, при размере контекста на 27 % меньше. Наши результаты согласуются с этим выводом: OntoRAG не только сохраняет, но и улучшает

качество благодаря более целенаправленному извлечению фрагментов через графовые связи. Улучшение NDCG@5 с 0.9435 до 0.9506 подтверждает, что онтологическое расширение повышает качество ранжирования.

Обнаруженные нами результаты по BFS-шуму при избыточном расширении графа согласуются с наблюдениями Edge и соавт. [13] в контексте GraphRAG. Решение в виде семантической фильтрации и приоритизации типов связей может быть полезно для аналогичных графовых систем.

Настоящее исследование имеет ряд ограничений, которые следует учитывать при интерпретации результатов. Корпус из 30 фрагментов и тестовая выборка из 18 вопросов являются малыми, а статистическая значимость различий между конфигурациями не подтверждена формальными тестами, такими как парный t-тест. На большем корпусе результаты могут отличаться.

Замена генерации извлечением предложений, хотя и позволяет изолировать влияние архитектурных компонентов, не отражает реальное поведение системы с полноценной языковой моделью, которая может по-разному реагировать на обогащённый и стандартный контекст. Эвристический классификатор на основе ключевых слов и плотности сущностей является упрощённым, и обучаемая модель, тренированная на размеченных данных, могла бы обеспечить более точную маршрутизацию.

Результаты получены в одной предметной области — математическом анализе, - и их перенос на другие области (медицина, право, техническое обслуживание) требует отдельного исследования. Кроме того, три из 18 вопросов показывают незначительное снижение метрик при маршрутизации на OntoRAG, что указывает на потенциал для дальнейшего улучшения классификатора.

Заключение

В ходе диссертационной работы исследованы возможности повышения эффективности архитектуры генерации с дополненной выборкой за счёт интеграции онтологического слоя и механизма адаптивной маршрутизации запросов. Проведён анализ современных подходов к развитию RAG-систем, включая онтологические и графовые методы, на основе которого была разработана модульная архитектура OntoRAG v3.2. Архитектура включает уровни извлечения, онтологического расширения, ранжирования и маршрутизации, а также поддерживает независимое тестирование компонентов и воспроизводимость экспериментов. Для определения

целесообразности применения онтологии реализован эвристический классификатор запросов, учитывающий наличие отношений, плотность сущностей и графовую связность. Программный прототип реализован на языке Python без внешних зависимостей и протестирован на корпусе по математическому анализу с использованием пяти метрик качества и серии абляционных экспериментов.

Экспериментальные результаты показали, что применение онтологического слоя при наличии семантической фильтрации и адаптивного ре-ранжирования позволяет улучшить качество генерации относительно базовой архитектуры RAG. OntoRAG продемонстрировал рост средних значений ROUGE-L и BLEU, а AdaptiveRAG обеспечил дополнительное повышение качества и положительный эффект на большинстве типов запросов, особенно на вопросах, связанных с межпонятийными отношениями и обобщением информации. Полученные результаты подтверждают, что онтология наиболее эффективна при избирательном использовании, а адаптивная маршрутизация позволяет снизить влияние нерелевантного графового контекста. Разработанный фреймворк может быть перенесён на другие предметные области и служит основой для дальнейших исследований гибридных архитектур RAG с использованием полноценных языковых моделей, обучаемых классификаторов и более сложных методов графового поиска.

Список литературы

1. Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., Wang, H. Retrieval-Augmented Generation for Large Language Models: A Survey // arXiv preprint arXiv:2312.10997v5. - 2024.
2. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., Kiela, D. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks // NeurIPS 2020. - Т. 33. - С. 9459–9474.
3. Borgeaud, S., Mensch, A., Hoffmann, J. и др. Improving Language Models by Retrieving from Trillions of Tokens // ICML. -PMLR 162. -2022. -С. 2206–2240.
4. Sharma, K., Kumar, P., Li, Y. OG-RAG: Ontology-Grounded Retrieval Augmented Generation For Large Language Models // arXiv preprint arXiv:2412.15235. -2024.

5. Kiss, C., Nagy, M., Szilágyi, P. Max–Min semantic chunking of documents for RAG application // Discover Computing. -2025. -T. 28. -C. 117.
6. Peng, B., Zhu, Y., Liu, Y., Bo, X., Shi, H., Hong, C., Yan, H., Li, J. Graph Retrieval-Augmented Generation: A Survey // arXiv preprint arXiv:2408.08921. -2024.
7. Ludwig, H., Schmidt, T., Kühn, M. An ontology-based retrieval augmented generation procedure for a voice-controlled maintenance assistant // Computers in Industry. -2025. -T. 169. -C. 104289.
8. Toro, S., Anagnostopoulos, A. V., Bello, S. M. и др. Dynamic Retrieval Augmented Generation of Ontologies using Artificial Intelligence (DRAGON-AI) // Journal of Biomedical Semantics. -2024. -T. 15. -C. 13.
9. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., Polosukhin, I. Attention Is All You Need // NeurIPS 2017. -T. 30. -C. 5998–6008.
10. Gruber, T. R. A Translation Approach to Portable Ontology Specifications // Knowledge Acquisition. -1993. -T. 5, No 2. -C. 199–220.
11. Guarino, N., Oberle, D., Staab, S. What Is an Ontology? // Handbook on Ontologies. -Springer. -2009. -C. 1–17.
12. Es, S., James, J., Espinosa-Anke, L., Schockaert, S. RAGAs: Automated Evaluation of Retrieval Augmented Generation // EACL 2024. -C. 150–163.
13. Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Larson, J. From Local to Global: A Graph RAG Approach to Query-Focused Summarization // arXiv preprint arXiv:2404.16130. -2024.
14. Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., Manning, C. D. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval // arXiv preprint arXiv:2401.18059. -2024.

15. Ji, Z., Lee, N., Frieske, R. и др. Survey of Hallucination in Natural Language Generation // ACM Computing Surveys. -2023. -T. 55, No 12. -C. 1–38.
16. Heinbockel, J. H. Introduction to Calculus. Volume II. -Old Dominion University, 2012.
17. Robertson, S. E., Zaragoza, H. The Probabilistic Relevance Framework: BM25 and Beyond // Foundations and Trends in IR. -2009. -T. 3, No 4. -C. 333–389.
18. Salton, G., Buckley, C. Term-weighting Approaches in Automatic Text Retrieval // Information Processing & Management. -1988. -T. 24, No 5. -C. 513–523.
19. Lin, C.-Y. ROUGE: A Package for Automatic Evaluation of Summaries // ACL-04 Workshop. -2004. -C. 74–81.
20. Papineni, K., Roukos, S., Ward, T., Zhu, W.-J. BLEU: a Method for Automatic Evaluation of Machine Translation // ACL 2002. -C. 311–318.
21. Järvelin, K., Kekäläinen, J. Cumulated Gain-Based Evaluation of IR Techniques // ACM TOIS. -2002. -T. 20, No 4. -C. 422–446.
22. McGuinness, D. L., van Harmelen, F. OWL Web Ontology Language Overview // W3C Recommendation. -2004.
23. Brown, T. B., Mann, B., Ryder, N. и др. Language Models are Few-Shot Learners // NeurIPS. -2020. -T. 33. -C. 1877–1901.
24. Touvron, H., Lavril, T., Izacard, G. и др. LLaMA: Open and Efficient Foundation Language Models // arXiv preprint arXiv:2302.13971. -2023.